

Gaffer's Clipboard - Match Ratings Methodology

Executive Summary

The quantification of a football performance into a single number from 0.0 to 10.0 has long been a complex and controversial process. At its core, it is an exercise in dimensionality reduction - distilling a web of complex and often interconnected statistics down to a single number. Industry-standard algorithms, such as those employed by platforms like *Sofascore* and *Fotmob*, have made significant strides in this area. However, fans frequently view them critically, as these systems carry inherent biases and operate as 'black boxes' with little transparency into how they actually weigh and analyse data.

The Gaffer's Clipboard Analytics Engine seeks to address these flaws by providing a transparent, mathematically rigorous 0.0 to 10.0 scale. While it has its own limitations (relying on statistics extracted directly from EA FC/FIFA screens), the engine operates completely offline. It feeds match telemetry through a strict pipeline involving data standardisation, principal component analysis, and contextual heuristics. By evaluating player performance solely on what unfolds on the virtual pitch, the engine offers users a clear, unbiased view of performance, fostering trust and confidence in tactical decisions and long-term tracking.

I. Introduction and Philosophy

In today's modern world, there is an increasing emphasis on viewing things through the lens of data and statistics. Whether assessing a company's performance over a financial quarter, a student's academic abilities in school, or the supposed success of a government, organisations and individuals increasingly seek the ability to strip away human error and bias to view performance through an objective lens. Moreover, the field of sports is no different. Whilst there are a few other similar fields, like politics, few others come close to the level of tribalism and, therefore, inherent biases found in sports and amongst their stakeholders. Along with how prevalent sports can be in our daily lives and the growing economic value they create, this explains why sports teams, fans, and others involved are increasingly focused on quantifying sports into objective data.

However, this is harder than may be initially thought, and harder with some sports than others. For example, analysing the performance of a sprinter is easy; there is only one real metric that matters - their time. Moreover, if one wants to go deeper, with cameras and basic timing

technology, it is relatively easy to track their top sprint speed, their acceleration over the first x meters, and so on. Team sports, on the other hand, are a different ball game, and football more than most. In the NBA, for example, it is somewhat easy to determine who did well in a game without watching it, simply by looking at simple metrics: points, assists, rebounds, steals, turnovers, etc. However, whilst football may have similar metrics: goals, assists, passes, tackles, shots, etc, these are nowhere near as informative. Whilst in basketball, fans may debate about who is the best ever by looking at career averages like points or field goals, the only statistic in football that comes somewhat close to that level of prominence in these debates is goals scored, and that is only one aspect of a player, and an even smaller aspect if that player is not a striker.

Part of why football is so much harder to quantify is that luck is a more significant factor in it than in other team sports. With such long matches and yet such small scorelines, it is common to see players and teams perform well, yet draw with or lose to teams supposedly performing much worse, simply because they had one moment of luck. Another is the sport's complexity and the expectations placed on players. Whilst a golfer may only be expected to put the ball towards the hole, a single footballer may be expected to consistently pass the ball to his teammates successfully (but if he is making safe passes too much that may be seen as bad), tackle opposing players and intercept their passes, cover a lot of ground across the pitch, create chances for their teammates, and get in good positions to shoot themselves. This is before one looks at aspects of their performance that are even harder to quantify in a single number. Take pressing: the action of a player or team applying pressure to the opponent whilst they have the ball, attempting to force turnovers, minimise time on the ball, and win possession. It is incredibly hard to scale how well a player or team performs at this into a single number.

Therefore, reducing all aspects of a player's performance, a matrix of spatial, temporal and event-based telemetry, into one unified player rating poses significant mathematical and epistemological challenges. The engine evaluates a variety of data types expressed in different ways - some unbounded, others not normally distributed - and attempting to quantify them on a finite scale.

At least amongst fans, a few apps and their commercial algorithms dominate this space (like *Sofascore*, *Fotmob* and *Whoscored*). These are increasingly significant in shaping how fans view players, influencing public perception and even professional scouting. Fans who have not watched a game will still often confidently form an opinion on how a player played based on a single number. However, at the same time, a lot of fans are noticing the serious deficiencies and drawbacks of these models, from their strong attacking bias (teams of the week and of tournaments generated by these services often do not represent a realistic team, usually packed too full of attacking players), to their lack of transparency (serious frustration comes from the black box nature of these algorithms, fans not being allowed to understand the inputs or how the algorithm works, only the output). The so-called "eye test" still is, and always will be,

important in evaluating a performance, and fans often express annoyance when mathematical ratings do not reflect what they have watched with their own eyes.

Performing the previously mentioned analyses, not on real football matches but on simulated ones via a video game, is even more complicated. EA FC's simulation and the stats it generates from it have their own drawbacks. Whilst the video game has its own match rating system built in, it is deeply flawed. The in-game algorithm evaluates players in a tactical vacuum, heavily over-indexing on raw goal involvements while routinely ignoring positional responsibilities. For example, a fullback explicitly instructed to stay back and defend will consistently receive artificially low match ratings for failing to register attacking output. Furthermore, the game's temporal scaling is highly volatile; an 85th-minute substitute can drastically inflate their match score with a mere handful of successful passes, breaking managerial immersion. These limitations are exacerbated by how the Gaffer's Clipboard application works - to make it easier to use, and the data logging process smoother and simpler, only 16 stats are collected directly from each player's performance, along with 16 stats collected on each team's collective performance in the match. Additionally, the goal of developing a useful, lightweight desktop application for users on all kinds of computers discourages the use of heavy machine learning libraries or massive cloud servers. Instead, the aim was to extract heavier aspects of the algorithm into a purely development-only phase, storing their outputs with the program, and then, when analysing a user's own data generated as they use the app on their own computer, simply using pure maths, lightweight `numpy` arrays and contextual heuristics. By decoupling heavy data science from the live scoring engine, the application delivers enterprise-grade analytical depth locally and instantaneously, without any external dependencies.

Before the engine can process the match data, it must confront the strict limitations of this native telemetry API. Real-world football analytics engines (such as those powered by *Opta*) benefit from hundreds of granular micro-events, including "Key Passes," "Interceptions," and "Progressive Carries." The Analytics Engine, however, is constrained by the rigid, 16-metric outfield matrix provided by the game's post-match extraction.

Because the engine lacks advanced spatial data, it cannot measure a player's creativity purely through counting stats. To bridge this gap, the algorithm synthesises a 17th metric - Expected Threat (xT) - derived from a combination of passing, dribbling, and physical exertion parameters. This final 17-element array serves as the absolute boundary of the engine's worldview. Everything the algorithm understands about a player's performance must be mathematically deduced from this highly restricted telemetry matrix.

Deep Dive: The Outfield Telemetry Matrix >

01a - The Outfield Telemetry Matrix

1. The Native API Constraints

The Analytics Engine operates on a strictly bounded data diet. The native extraction tool captures only 16 base metrics for outfield players. These metrics are heavily biased toward traditional counting stats and lack the granular transition data (e.g., interceptions, progressive carries, key passes, aerial duels) standard in modern data science.

To standardise the evaluation of playing time, all volume-based counting stats are normalised "Per-90" (p90) rate before entering the matrix.

2. The 16-Element Native Array

The raw telemetry matrix is partitioned into four distinct tactical categories:

Attacking & Output:

1. goals_p90
2. assists_p90
3. shots_p90
4. shot_accuracy (%)

Possession & Distribution:

5. passes_p90
6. pass_accuracy (%)
7. dribbles_p90
8. dribble_success_rate (%)

Defensive Actions:

9. tackles_p90
10. tackle_success_rate (%)
11. possession_won_p90
12. possession_lost_p90

Physicality & Discipline:

13. distance_covered_p90
14. distance_sprinted_p90
15. fouls_committed_p90
16. offsides_p90

3. The 17th Dimension: Synthetic Expected Threat (xT)

Because the native 16-element array completely lacks spatial progression metrics (e.g., measuring the difference between a safe sideways pass and a line-breaking

pass), the engine generates a 17th synthetic feature: **Expected Threat** (`xt_bonus_p90`).

This derived metric acts as a mathematical proxy for ball progression and spatial dominance. By combining and weighting elite passing volume, dribbling volume, and sprint distance, the engine creates a synthetic indicator of how effectively a player moved the ball into dangerous areas.

This finalised 17-element array (T_{17}) serves as the sole input vector for the downstream PCA eigen-decomposition and positional heuristic pipelines.

Despite the existing limitations, the goal of this algorithm was to eliminate the black box commonly used by industry-standard services. The architecture takes messy OCR-extracted telemetry data and employs a transparent, purely mathematical approach to distil it into a single player rating. Similar to professional alternatives, there was a strong emphasis on context - considering positional variance, intelligent tactical roles, team performance, and match configurations. Ultimately, this algorithm aims to synthesise these complex contextual variables into a single, transparent metric, transforming raw telemetry into a definitive and unbiased reflection of true on-pitch value.

II. Data Pre-Processing & Normalisation

Every statistical model, regardless of its complexity, is bound by one fundamental law: the quality of its output is entirely dependent on the quality of its inputs. After validating the extracted telemetry, the application verifies that the values fall within expected ranges before routing them through the algorithm and applies common-sense rules, such as goals cannot be greater than shots, and assists cannot be greater than passes. However, in its rawest form, this extracted telemetry is simply a collection of isolated event counts, inherently devoid of context. Feeding this unadjusted data directly into a complex scoring algorithm would create highly suspect outputs, as it treats all events as equal, regardless of the opportunities provided to the player.

In football, this concept of opportunity is inextricably linked to time. Evaluating a player purely on their gross accumulation of stats heavily skews the data in favour of those who complete a full match, whilst punishing those who are substituted on or off. These raw counts hide efficiency; a player registering 15 passes in 90 minutes has performed quite poorly in that area, whilst a player who has achieved the same number in a 15-minute cameo has performed quite well. To accurately judge a performance, the data must reflect a rate of action rather than a raw sum. Across football statistics, this is usually expressed as a rate per 90: the number of actions the player would have completed had he played 90 minutes. Statistics shown across a season, such as goals, are often presented this way: goals per 90. This method shows how well a

player has performed relative to the opportunities they have had, accounting for injuries and adding parity to player comparisons.

When translating this logic to a video game simulation like EA FC, the temporal problem doubles. Unlike real-world football, simulation telemetry is fixed to two clocks: the 90 minutes of the football match and a customisable half-length game setting. This setting sets the number of real-world minutes each half takes, which can range from 4 to 20 minutes, and the in-game 45 minutes scales accordingly. The problem this poses is that, with more real-world time to play, and for instance press the pass button more times, the volume counts will be higher in longer half-length matches. To ensure mathematical fairness and cross-match comparability, the Analytics Engine must first pass all raw OCR-extracted data through a strict pre-processing pipeline. This phase standardises the volume metrics and masks low-volume anomalies, ensuring the foundational data is perfectly level before any scoring or positional weighting is applied.

Temporal Standardisation

To address the half-length problem, the engine introduces a baseline half-length setting that scales all cumulative volume stats, regardless of their original half-length. This baseline is set at 10 minutes, and all cumulative volume stats (passes, shots, tackles, etc.) are scaled to it. The maths to achieve this is simple, multiplying the raw stat by the ratio of the baseline half-length to the real half-length ($X_{adj} = X_{raw} \times \left(\frac{H_{base}}{H_{match}}\right)$). For example, if a user is playing 4-minute halves, then a midfielder completing 20 passes is playing quite well. However, if this were directly compared to a 10-minute half (where midfielders playing the full match usually average around 40 passes) without standardisation, it would look quite poor. To scale this volume up, the algorithm passes the metric through the standardisation formula ($20 \times \left(\frac{10}{4}\right) = 50$), and it is converted to its 10-minute half equivalent: 50 passes. This performance can now be fairly compared against a performance from a match with 10-minute halves.

Handling Substitute Volatility (Bayesian Smoothing)

While temporal standardisation fixes EA FC's custom clock, the engine must still apply the standard fix: per-90 normalisation. However, simply converting these adjusted stats into a per-90 rate introduces a new mathematical hazard: substitute volatility. If an 88th-minute substitute comes on and takes a single shot on target in two minutes of game time, this standard formula would give a per 90 rate of an absurd 45 shots per game. If the program fed that inflated rate directly into the scoring matrix, it would break the algorithm and generate a 10.0 rating for a two-minute cameo.

To prevent this, the engine demands proof of sustainability. It applies a statistical technique known as Bayesian smoothing, which acts as a mathematical anchor. By blending the player's observed stats with a historical positional average, the engine ensures that a player's rating is

closely tied to the mean (generating a standard 6.0 rating) when they first step onto the pitch. The fewer minutes a player has on the field, the stronger this anchor holds, meaning their calculated output is largely dictated by historical expectations rather than their brief cameo. Therefore, if they played for only a few minutes, it is highly unlikely their rating would be far from the standard 6.0. To break away from this baseline and earn an elite (or abysmal) rating, the player must physically accrue enough minutes - and consistent statistical output - to overpower the positional average.

Deep Dive: Bayesian Smoothing Architecture >

02b - Bayesian Smoothing Architecture

1. The Mathematical Hazard of Small Samples

As outlined in the theoretical methodology, extrapolating small-sample simulation data (such as 1 shot in 2 minutes) into a "per 90" rate relies on a linear assumption that fundamentally breaks at such extremes. The danger of this is not just the inflated number itself, but how it interacts with downstream calculations. When an unanchored, mathematically absurd extrapolation - such as 45.0 shots per 90 - enters a standard deviation (Z-score) matrix, it registers as a multi-sigma outlier. This artificially warps the player's dimensional weight and completely bypasses positional constraints, mathematically guaranteeing a maximum match rating of 10.0 for a brief cameo with very little impact on the actual game.

2. The Shrinkage Anchor

To resolve this, the engine utilises Bayesian smoothing. In this context, it operates as a statistical shrinkage technique that pulls low-confidence observations (those with small sample sizes) toward a high-confidence prior (historical averages).

The conceptual blending formula is as follows:

$$r_{smoothed} = \frac{M}{M + d} r_{obs} + \frac{d}{M + d} r_{prior}$$

- M : Minutes actually played.
- r_{obs} : The observed statistical rate.
- d (**Dummy Anchor**): A constant representing the weight of historical expectation ($d = 30.0$).
- r_{prior} : The historical expected rate for that specific position.

The constant d dictates the critical "pivot point" of the algorithm - the exact minute mark where the mathematical weight of a player's live observed performance (M)

equals the weight of the historical expectation (d). During development, this constant was rigorously calibrated against alternative thresholds to find the optimal balance between variance protection and statistical freedom.

- **The Reactive Trap ($d = 20.0$):**

If the anchor is set too low, the engine releases its grip on the prior too quickly. At $d = 20$, a player substituted on in the 70th minute reaches the 50/50 pivot point by the final whistle. This allows late-game variance to dictate their rating heavily. A single lucky bounce or a brief five-minute spell of dominance would mathematically outweigh the anchor, resulting in wildly inflated ratings for players who barely influenced the match's overarching narrative.

- **The Suffocation Trap ($d = 45.0$):**

Conversely, setting the anchor too high mathematically suffocates legitimate performances. At $d = 45$, the 50/50 pivot point is not reached until a player completes a full half of football. If a manager brings on a substitute at half-time, and that player delivers 45 minutes of elite tactical execution, a $d = 45$ anchor would brutally drag their rating back toward a baseline 6.0, refusing to reward a genuinely match-altering performance.

- **The Impact Sub Equilibrium ($d = 30.0$):**

Setting $d = 30.0$ establishes a perfect mathematical equilibrium. In football analytics, 30 minutes represents the classic "impact sub" window (a 60th-minute substitution) and is widely considered the minimum threshold required to gauge tactical integration. By hardcoding $d = 30.0$, the algorithm enforces a highly accurate sliding scale of trust:

- **The Cameo ($M = 5$):** The anchor holds firm, heavily trusting the historical prior ($\sim 86\%$) to prevent extreme variance from a few isolated touches.
- **The Equilibrium ($M = 30$):** The live data and the historical prior carry an exact 50/50 weight, demanding that an impact sub sustains their performance to earn an elite rating.
- **The Full Match ($M = 90$):** The player fully overpowers the anchor, with the algorithm trusting their live observed data (75%) while retaining a minor 25% shrinkage to account for standard match-to-match simulation variance.

3. The Logic of Priors (r_{prior})

For Bayesian smoothing to function logically, the algorithm cannot use a blanket prior for all statistics. The engine must bifurcate the prior (r_{prior}) based on the fundamental nature of the telemetry being evaluated.

Rare, Discrete Events ($r_{prior} = 0.0$)

Metrics such as goals, assists, and shots represent highly valuable, discrete, and inherently rare events. For these `rare_cols`, the prior must be strictly set to 0.0. If the algorithm used a historical average - for example, if a striker historically averages 0.6 goals per game - the blending formula would falsely inject "ambient" fractional goals into a substitute's baseline rate simply for stepping onto the pitch. In a mathematically rigorous engine, decisive match-altering actions cannot be assumed; they must be earned entirely from scratch through live telemetry.

Continuous, Volume Events ($r_{prior} = \mu_{historical}$)

Conversely, cumulative metrics such as passes, tackles, and distance covered represent continuous tactical execution. Even when a player is not actively on the ball, they are occupying space, pressing, and moving within a system. For these `volume_cols`, the prior dynamically queries the offline `means_stds` dictionary to fetch the exact historical mean (μ) for the active positional role.

If we used a 0.0 prior for these continuous events, the algorithm would incorrectly assume an 80th-minute defensive midfielder actively refused to pass or tackle, artificially dragging their efficiency scores toward zero. By anchoring volume metrics to the historical positional mean, the engine gives the player the "tactical benefit of the doubt"—assuming they are fulfilling the baseline responsibilities of their role at an average rate, until their active telemetry proves they are over- or under-performing.

4. Algorithmic Implementation (`_apply_bayesian_smoothing`)

Within the `MatchRatingsService`, this mathematical foundation is executed with a strict focus on application stability, efficient memory allocation, and defensive programming. Because this engine is designed to run locally on a user's desktop without heavy data-science libraries like `pandas`, the implementation relies entirely on highly optimized native Python dictionaries. Working through the algorithm, the engine receives a dictionary of `normalized_metrics` (stats already adjusted for half-length) and routes them through a bifurcated processing pipeline.

The Rare Events Matrix (Dictionary Comprehension)

For discrete match-altering events (`rare_cols` such as goals, assists, and shots), the prior is mathematically mandated to be 0.0. Because the prior is zero, the entire right side of the conceptual blending formula is neutralized.

To maximize runtime efficiency, the engine evaluates these metrics using a rapid dictionary comprehension. The algebraic reduction is simplified even further, requiring

no database queries:

$$X_{p90} = \left(\frac{X}{M + d} \right) \times 90.0$$

This elegantly applies the heavy shrinkage anchor to low-minute players without the computational overhead of querying positional averages.

The Volume Matrix and Safe Dictionary Traversals

For continuous tactical events (`volume_cols` such as passes, tackles, and distance covered), the engine must fetch the exact historical mean (μ) for the player's specific tactical role.

In a live application environment, querying external datasets is a common point of failure (e.g., a newly introduced tactical role might be missing from the database). To guarantee uninterrupted execution, the engine utilizes a defensive, nested `.get()` traversal of the offline `means_std` dictionary:

```
col_stats = self.means_std.get(pos, {}).get(
    f"{col}_p90",
    {"mean": 0.0, "std": 1.0}
)
league_average_p90: float = col_stats.get("mean", 0.0)
```

This architecture ensures that if a position or metric is missing, the application defaults safely to a 0.0 mean rather than throwing a `KeyError` and crashing the analysis.

Once the prior is safely retrieved, the algorithm computes the `dummy_stat`. This is the literal Python representation of $r_{prior} \times (d/90)$, calculating exactly how many actions a statistically average player *should* have completed in 30 minutes. This fractional volume acts as the literal anchor weight dropped onto the scales before executing the final reduction.

Absolute Stability and Divide-by-Zero Safety

Executing the standard conceptual Bayesian formula requires the engine to first calculate the observed rate (r_{obs}) by dividing raw volume by actual minutes played (X/M).

However, calculating X/M introduces a critical systemic hazard. If the EA FC telemetry glitches, or if a user accidentally logs a player with 0 minutes played,

executing $X/0$ will immediately crash the application with a `ZeroDivisionError`.

By executing the algebraically reduced formula:

$$X_{p90} = \left[\frac{X + \text{dummy_stat}}{M + d} \right] \times 90$$

The engine never divides by M . Because the denominator is strictly $(M + d)$, and d is hardcoded to 30.0, the denominator can never equal zero. This mathematically identical reduction achieves two things simultaneously: it calculates highly stable, small-sample-proof metrics, and it structurally guarantees a crash-proof runtime function.

Mathematical Proof: Equivalence of the Reduced Formula >

To prove that the optimized Python implementation mathematically matches the conceptual Bayesian formula, we map the variables as follows: X = raw volume, M = minutes played, $d = 30.0$.

Step 1: The Conceptual Formula

$$r_{smoothed} = \left(\frac{M}{M + d} \right) r_{obs} + \left(\frac{d}{M + d} \right) r_{prior}$$

Step 2: Substitute r_{obs} for its raw mathematical components

The observed per-90 rate (r_{obs}) is calculated as $(X/M) \times 90$. Substituting this in:

$$r_{smoothed} = \left(\frac{M}{M + d} \right) \left(\frac{X}{M} \times 90 \right) + \left(\frac{d}{M + d} \right) r_{prior}$$

Step 3: Cross-Cancel and Align Denominators

In the first term, the M in the numerator and the M in the denominator cancel each other out:

$$r_{smoothed} = \frac{X \times 90}{M + d} + \frac{d \times r_{prior}}{M + d}$$

Because both terms now share the exact same denominator $(M + d)$, they can be combined into a single fraction:

$$r_{smoothed} = \frac{90X + d \cdot r_{prior}}{M + d}$$

Step 4: Align with the Python Implementation

The Python code structures the formula to multiply by 90 at the very end to format the rate:

$$X_{p90} = \left[\frac{X + r_{prior} \times \left(\frac{d}{90} \right)}{M + d} \right] \times 90$$

Distributing the $\times 90$ into the numerator yields:

$$X_{p90} = \frac{90X + \left(r_{prior} \times \frac{d}{90} \times 90 \right)}{M + d}$$

The 90 and $/90$ on the right side cancel out, leaving the final equation:

$$X_{p90} = \frac{90X + d \cdot r_{prior}}{M + d}$$

Conclusion: The formulas are algebraically identical. The X_{p90} reduction safely calculates the exact same shrinkage rate without ever dividing by M .

III. Live Feature Engineering & Standardisation

With the raw telemetry successfully cleaned, standardised, and insulated against substitute volatility, the foundational data entering the algorithm is mathematically stable. However, even after this pre-processing, these stats cannot be fed directly into a rating matrix. In their current form, they lack two critical mathematical properties: dimension and equivalence.

Football is an inherently multidimensional sport, and evaluating a player requires balancing their offensive output, defensive contributions, and possession retention. Raw counting stats offer no way to compare these differing actions; one cannot simply add "3 tackles" to "50 passes" to deduce a match rating. Is 50 a good number of passes? And if it is, is 50 a similarly good number of tackles, or shots, for example.

To resolve this problem of comparing apples to oranges, the Analytics Engine executes a rigorous live feature engineering pipeline. This phase synthesises advanced metrics using mathematical estimations and proxies to replace missing spatial context. It then converts all telemetry into a universally comparable scale and enforces strict minimum-volume thresholds to ensure that a player's calculated efficiency is fully earned.

The Expected Threat (xT) Proxy

In real-world football analytics, the Expected Threat (xT) metric, like related metrics such as Expected Goals (xG), is calculated using precise X/Y pitch coordinates. Expected Threat uses these to measure how much a player's actions increase the team's probability of scoring - how much threat they generate. It does this by mapping a player moving the ball from a low-value zone (such as around the halfway line) to a high-value zone (such as in or around the opponent's penalty box). However, the data extracted from EA FC post-match stats screens

entirely lacks spatial context. In the game's raw dataset, a five-yard sideways pass between two centre-backs is indistinguishable from a 40-yard line-breaking pass from a playmaker. This discrepancy is made worse by the fact that no similar metric, such as Expected Assists (xA), is collected, which would show the quality of the chances created, regardless of whether the playmaker's teammates finished them. Therefore, the algorithm would have no way to reward a playmaker who generated numerous great chances but played with a striker who could not finish them. Without an xT metric, the algorithm would inherently reward passive, stat-padding possession just as highly as dynamic, attacking football.

To bridge this spatial void, the Analytics Engine synthesises an xT proxy by combining available metrics to approximate Expected Threat. This heuristic is grounded in real-world sports science - research from the CIES Football Observatory and tracking data from UEFA Euro 2016 demonstrate that while total distance covered has little correlation to match success, high-intensity sprinting is statistically linked to positive goal differentials and is most commonly seen among attacking players.^{[1][2]} Furthermore, industry analysis shows that elite teams combine this physical intensity with high passing volume to generate a true attacking threat.^[3] By mathematically binding a player's sprint ratio to their passing efficiency, the engine accurately isolates box-to-box midfielders, overlapping fullbacks, and direct wingers who drive progressive, line-breaking possession.

Crucially, this proxy cannot scale linearly. A player increasing their passing volume from 20 to 40 accurate passes usually signifies a massive increase in active match involvement and threat. However, a player increasing from 80 to 100 passes is often just recycling safe possession in a deep defensive block. To reflect this reality, the engine applies a natural logarithm to the passing data. Doing so ensures that the xT bonus highly rewards players for establishing a baseline of dynamic progression, but applies mathematical diminishing returns to prevent players from artificially inflating their rating through sterile, non-threatening possession.

Deep Dive: Expected Threat (xT) Synthesis >

03a - Expected Threat Synthesis

1. The Algebraic Foundation

To synthesise a proxy for Expected Threat (xT) without spatial coordinate data, the `MatchRatingsService` uses a heuristic formula that mathematically links a player's physical intensity to their technical execution. The algorithmic formula is defined as:

$$xT_{bonus} = 0.25 \times \left(\frac{D_{sprint}}{D_{total}} \right) \times \ln((A_{pass} \times V_{pass}) + 1)$$

- D_{sprint} : Distance sprinted per 90.
- D_{total} : Total distance covered per 90.

- A_{pass} : Pass completion percentage (expressed proportionally).
- V_{pass} : Total passes attempted per 90.

2. The Physicality Multiplier and Scalar Calibration

The first half of the equation, $\left(\frac{D_{sprint}}{D_{total}}\right)$, establishes the player's physical intent. By converting raw sprint distance into a ratio of total distance covered, the engine isolates players who are aggressively breaking lines rather than passively jogging in a defensive shape.

Crucially, this ratio is heavily throttled by a hardcoded 0.25 scalar. Because this metric is an artificial proxy being appended to a highly regulated, variance-sensitive Z-score matrix, the scalar was rigorously calibrated to ensure the xT_{bonus} functions strictly as a marginal reward for dynamic play, rather than a primary driver that could mathematically overpower a player's core rating.

During development, this scalar was calibrated against elite simulated outputs to find the mathematical "tiebreaker" equilibrium:

- **The Overpowering Trap (Scalar = 1.0):**

If an elite playmaker registers a 12% sprint ratio (0.12) alongside 70 accurate passes ($\ln(71) \approx 4.26$), an unscaled formula yields a raw bonus of +0.51.

Appending +0.51 directly to a Z-score is mathematically disastrous; it effectively artificially inflates a player's rating by half a standard deviation. This would allow a winger with terrible crossing accuracy to achieve a Man of the Match rating purely by sprinting and recycling safe passes.

- **The Invisible Trap (Scalar = 0.05):**

Conversely, setting the scalar too low yields a bonus of roughly +0.02. This is mathematically negligible and would routinely be erased by standard floating-point rounding during the final 1-10 match rating conversion, defeating the purpose of the proxy entirely.

- **The Tiebreaker Equilibrium (Scalar = 0.25):**

Applying the 0.25 scalar compresses the world-class playmaker's raw bonus down to roughly +0.12 to +0.15 Z-score units. This is the optimal mathematical band. It is not enough to save a fundamentally poor performance, but it is enough to break a tie. If two midfielders both achieve a +1.0 Z-score based on their core efficiency, the engine correctly awards the marginally higher final match rating to the player who generated higher physical threat.

3. The Technical Base and Logarithmic Restraint

The second half of the equation, $\ln((A_{pass} \times V_{pass}) + 1)$, evaluates technical execution. Multiplying A_{pass} by V_{pass} calculates the exact volume of successful passes.

Wrapping this product in a natural logarithm ($\ln$) applies strict mathematical diminishing returns. A player who completes 30 passes receives a significant mathematical spike, reflecting active match involvement. However, a player who completes 90 passes receives a severely diminished marginal return, preventing them from exponentially farming the bonus through sterile, non-threatening possession around the backline.

Mathematically, the $+1$ constant is a critical structural necessity. If a player completes 0 passes, $\ln(0)$ results in negative infinity, which would instantly corrupt the rating matrix. By appending $+1$, the engine guarantees that the absolute minimum return is $\ln(1)$, which safely resolves to 0.0.

4. Algorithmic Implementation and Safety

Within the `MatchRatingsService`, this formula is executed strictly on data that has already passed through the temporal standardisation and Bayesian smoothing pipelines, to ensure small-sample volatility does not inflate the bonus.

```
def _calculate_xt_bonus(
    self,
    pass_accuracy: float,
    passes_p90: float,
    distance_sprinted_p90: float,
    distance_covered_p90: float,
) -> float:
    if distance_covered_p90 == 0:
        return 0.0

    return (
        0.25
        * (distance_sprinted_p90 / distance_covered_p90)
        * np.log(pass_accuracy * passes_p90 + 1)
    )
```

Beyond the logarithmic $+1$ safeguard, the engine includes a hard physical failsafe. If a telemetry glitch logs a player with 0.0 distance covered, calculating the sprint ratio would trigger a `ZeroDivisionError`. The explicit `if distance_covered_p90 == 0:` check intercepts this edge case, safely returning a 0.0 bonus and preserving application stability.

Live Z-Score Standardisation

Even with advanced proxies like Expected Threat, the telemetry remains dimensionally fragmented. A centre-back executing five successful tackles and a striker completing five shots on target have both registered the number 5, but the tactical weight, frequency and difficulty of those actions will be quite different. To calculate a singular match rating that encompasses all metrics appropriately, each metric must be converted to a universally comparable unit of measurement.

To achieve this dimensionality reduction, the Analytics Engine applies a mathematical technique known as Z-score standardisation. Rather than evaluating the player based on raw stat volumes, the algorithm measures their performance in standard deviations from the historical positional mean ($Z = \frac{X - \mu}{\sigma}$). Conceptually, a Z-score abandons the raw data; instead of asking 'how many passes a player completed', it asks 'how mathematically exceptional was this player's passing volume compared to a typical player in their position?'. This means that players are judged relative to the average players, giving their performances some context.

However, standard Z-score mathematics are context-blind - they only measure distance from the mean. If a player commits five fouls in a match, whilst the historical average is one, the standard formula will yield a massive, positive z-score, just like if the player had scored 4 goals. To handle this, negative telemetry stats, where the higher they are, the worse the performance (fouls committed, offsides and possession lost) - are intercepted and handled differently. For these specific metrics, an inverted formula is used ($Z = \frac{\mu - X}{\sigma}$), ensuring that high volumes of negative actions generate heavily negative Z-scores.

Once all metrics have been run through this pipeline, instead of being expressed as raw numbers, they are decimal numbers grouped around 0, and how far above and below 0 they are shows how much the player over- or under-performed in that metric. This is expressed in standard deviations, so the majority of values will be within one standard deviation from the mean, and will therefore be between -1 and 1. Stronger outlying performances may reach as far as three standard deviations, and therefore generate a Z-score between -3 and 3. This now means that all metrics are in a form that is easily comparable - one can now look at these 12 numbers and say whether a player did better or worse than expected in each statistic, backed up by context.

Volume Masking

Whilst Z-score standardisation normalises the telemetry to a comparable scale, it fails to address a fatal flaw in the efficiency metrics (pass accuracy, shot accuracy, dribble success rate, and tackle success rate). Because these metrics are percentages, they are very easily manipulated by low-volume events. If a defender, perhaps one who only played the second half, completed just two passes, and simple five-yard passes at that, they would achieve a 100%

pass accuracy, despite their passing contribution being effectively meaningless, and in no way comparable to a player with 100% pass accuracy after 40 passes. Nevertheless, the Z-score algorithm would treat this 100% as a massive outlier and assign an elite, highly positive Z-score to a metric that is, in this case, useless.

To protect the integrity of the rating, the Analytics Engine enforces strict volume masks. A player must meet an absolute minimum volume threshold to 'unlock' their efficiency Z-score. The algorithm mandates the following baselines before it will trust the accompanying accuracy metric:

- **5 passes**
- **3 dribbles**
- **2 shots**
- **2 tackles**

These specific thresholds were chosen as they represent the absolute minimum statistical footprint required to prove intentionality rather than coincidence. For example, a player might accidentally log a 100% tackle success rate from a single lucky bounce, but executing two successful tackles requires deliberate defensive timing.

If a player fails to meet these thresholds, the algorithm does not assign a negative rating. Instead, it intercepts the efficiency Z-score and explicitly overrides it to 0.0. Because 0.0 represents the exact mathematical mean in a standard deviation matrix, the engine elegantly neutralises the metric. So if the threshold is not met, the accuracy metric is reset to the baseline, and they are not rated up or down based on it.

IV. Offline Weight Generation: Dimensionality Reduction

While Z-score standardisation successfully establishes dimensional equivalence across all metrics, it introduces a secondary architectural challenge: establishing relative tactical value. A standardised passing metric and a standardised tackling metric are mathematically comparable, but these metrics do not hold equal weight in determining a player's overall match contribution. Furthermore, this valuation is inherently position-dependent; the statistical footprint of an elite centre-back is fundamentally different from that of an elite striker.

To resolve this, the Analytics Engine explicitly rejects arbitrary, hand-tuned weighting. Manually assigning tactical value introduces developer bias, inadvertently forcing the engine to judge a user's players against a rigid, subjective tactical philosophy rather than objective performance. Instead, positional the engine mathematically derives these weights. The algorithm executes this derivation entirely offline, leveraging a comprehensive historical dataset to calculate the weights outside of the live application environment. The resulting optimal weight matrices are

then hardcoded directly into the production engine, ensuring the live software remains computationally lightweight, perfectly objective, and mathematically rigorous.

The Tangled Stats Problem

Now, the biggest problem that comes up when attempting to derive and assign weights is the tangled stats problem: football statistics are often inherently tangled (highly correlated) with one another. For example, if a midfielder has a high number of passes, they are more likely to have more assists and a higher Expected Threat. Moreover, if a defender attempts a high number of tackles, they are also more likely to win a high number of possessions and, therefore, usually have a lower number of possessions lost. Assigning weights to these stats independently, without accounting for their relationships, creates a severe problem of multicollinearity (double- or triple-counting the exact same underlying performance trait). The engine would reward a player multiple times for doing the same thing.

The PCA Solution

To mitigate the multicollinearity hazard while maintaining tactical realism, the offline analytics pipeline utilises a hybrid mathematical approach: **Guided Principal Component Analysis (PCA)**. If standard PCA is left entirely unguided, it blindly extracts whichever metrics hold the highest historical variance. Doing so introduces a severe architectural flaw; the algorithm could theoretically assign a massive attacking weight to a Defensive Midfielder simply because shooting variance happened to be highly volatile in the dataset.

To prevent this, the engine pairs the developer's overarching tactical mandate (the Macro-Philosophy) with the algorithm's mathematical precision (the Micro-Derivation). First, the algorithm groups the 17 metrics into distinct tactical blocks: *Attacking*, *Passing*, *Dribbling*, *Defending*, *Safety*, *Workrate*, and *Tactical Discipline*. For specialist positions, the engine applies structural multipliers to these blocks *before* the PCA runs. For example, the philosophy for a Central Defensive Midfielder (CDM) artificially inflates the variance of the *Defending* and *Passing* blocks, while heavily suppressing *Attacking*. (The standard Central Midfielder operates as the engine's neutral baseline, where all blocks are weighted equally).

Each component represents a naturally occurring tactical profile defined by its own unique set of internal weights. Component 1 always captures the largest chunk of variance (e.g., 45%) and maps to the dominant trait of that position, while Component 2 captures the next largest chunk (e.g., 20%) for secondary traits. Because these components are mathematically orthogonal - meaning they have zero statistical correlation with one another - the algorithm successfully isolates a player's offensive output from their defensive output.

Finally, the engine must collapse these independent dimensions back into a single, usable set of multipliers. To generate the final positional weight for a specific stat, the algorithm looks at that stat's internal weight across all components. It scales each of those internal weights strictly

by the percentage of overall variance that the specific component explains, and then sums them. This method ensures the final Positional Weight Vector accurately implements the developer's tactical philosophy without double-counting the underlying telemetry.

Deep Dive: Principal Component Analysis and Weight Generation >

04b - PCA Algorithm and Generating Weights

1. Tactical Block Scaling

Before computing the covariance matrix, the engine receives an $N \times 17$ matrix of standardised Z-scores. However, it must restructure this raw data to account for two distinct architectural hazards: structural size bias and blind variance optimisation.

First, the 16 metrics are natively grouped into tactical blocks. Without scaling these blocks at all, blocks with more features (e.g., *Attacking* has 4 stats) artificially inject more variance into the dataset than smaller blocks (e.g., *Safety* has 1 stat), biasing the PCA to favour large blocks regardless of tactical importance.

Secondly, if standard PCA is left entirely unguided, it will blindly extract whichever metrics hold the highest variance. For example, there may be a lot of variance in defensive midfielders' shooting: in some matches, they may be pinned back, unable to get forward, whilst against weaker teams, they may have much more license to get forward. Standard PCA would therefore give CDM's shooting a massive weight simply because shooting variance is historically very volatile, even though shooting should not make up a big part of how good a CDM's performance was; aspects like passing and tackling are much more important.

To resolve this, the engine applies a **Guided Variance Transformation**. The developer defines a macro-philosophy dictionary that assigns a structural multiplier to each tactical block based on the specific positional mandate.

```
# The Baseline Configuration Schema
blocks = {
    "Attacking": [
        "goals_p90_z", "assists_p90_z", "shots_p90_z",
        "shot_accuracy_z"
    ],
    "Passing": [
        "passes_p90_z", "pass_accuracy_z"
    ],
    "Dribbling": [
        "dribbles_p90_z", "dribble_success_rate_z"
    ],
    "Safety": [
```

```

        "possession_lost_p90_z"
    ],
    "Defending": [
        "tackles_p90_z", "tackle_success_rate_z",
    "possession_won_p90_z"
    ],
    "Workrate_and_Threat": [
        "distance_covered_p90_z", "distance_sprinted_p90_z",
    "xt_bonus_p90_z"
    ],
    "Tactical_Discipline": [
        "offsides_p90_z", "fouls_committed_p90_z"
    ],
}
# Example: The CDM Macro-Philosophy
cdm_philosophy = {
    "Attacking": 1.0,
    "Passing": 2.5,
    "Dribbling": 0.6,
    "Safety": 1.4,
    "Defending": 2.4,
    "Workrate_and_Threat": 0.4,
    "Tactical_Discipline": 1.0,
}

```

When iterating through the Z-scores DataFrame, the engine identifies the number of stats within the current block (k) and queries the philosophy dictionary for the assigned macro-multiplier (W_{block}). It then scales every individual stat (X_z) within that block using the following formula:

$$X_{scaled} = X_z \times \frac{W_{block}}{\sqrt{k}}$$

By dividing by $\sqrt{k}$, the engine establishes a neutral baseline, ensuring the 4-stat *Attacking* block does not mathematically overpower the 1-stat *Safety* block simply by possessing more columns. Simultaneously, multiplying by W_{block} artificially dilates or compresses the variance of specific tactical zones *before* the PCA analyzes the matrix. For a CDM, artificially inflating the *Defending* and *Passing* variance forces the downstream eigen-decomposition to prioritize those exact traits when constructing the principal components.

2. Robust Covariance Estimation and the MCD Algorithm

Standard PCA relies on a traditional sample covariance matrix to map the relationships between metrics. However, the standard covariance formula is highly sensitive to extreme outliers because it calculates the variance using the arithmetic mean ($\bar{x}$) of the entire dataset (n):

$$C_{j,k} = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

In a video game simulation environment, football data frequently contains multi-sigma anomalies - such as a player scoring 4 goals from 0.5 xG, or a glitch in telemetry logging impossible physical stats. Because the standard formula squares the deviations from the mean, these extreme, unrepresentative performances heavily skew the matrix, warping the resulting principal components toward the outliers rather than the sustainable tactical baseline.

To establish algorithmic stability, the engine explicitly rejects the standard sample covariance in favour of the **Minimum Covariance Determinant (MCD)** estimator.

To understand MCD, visualise the dataset as a matrix where every row represents a single match performance. Standard covariance calculates variance using every single row. The MCD algorithm, dictated by the `support_fraction=0.98` parameter, is instructed to deliberately throw away 2% of those rows to find a "clean" core dataset. Mathematically, H represents this clean subset of surviving rows. Because the algorithm does not initially know *which* rows are the anomalies, it must hunt for them. It mathematically evaluates thousands of different combinations of keeping 98% of the rows and discarding 2%. For every potential combination (H), it calculates the determinant of its covariance matrix (C_H). Geometrically, the determinant measures spatial volume; a lower determinant means the data points are packed more closely together. The algorithm's objective is:

$$\min_H \det(C_H)$$

In plain English: The algorithm searches for the specific combination of rows (H) that yields the lowest possible spatial volume. By minimising the determinant, the algorithm actively isolates the tightest, most consistent 98% cluster of match performances. The 2% of rows that cause the data ellipsoid to stretch or distort significantly are identified as anomalies and discarded.

The robust covariance matrix (C_{MCD}) is then computed exclusively from the clean rows inside H , ensuring the downstream math is mathematically immunized against telemetry glitches.

$$C_{MCD} = \frac{1}{h-1} \sum_{i \in H} (x_i - \mu_{MCD})(x_i - \mu_{MCD})^T$$

The Python implementation utilizing `scikit-learn` :

```
# support_fraction=0.98 defines subset 'h' as 98% of 'n'
mcd = MinCovDet(random_state=42,
support_fraction=0.98).fit(scaled_pca_df) cov = mcd.covariance_
```

By substituting C_{MCD} in place of standard covariance, the engine ensures that the downstream eigen-decomposition is built strictly on sustainable, repeatable football logic, mathematically immunising the positional weights against telemetry glitches and statistical anomalies.

3. Eigen-Decomposition and Dimensional Truncation

With a robust covariance matrix (C_{MCD}) established, the algorithm executes eigen-decomposition. This is a linear algebra operation that rotates the standard coordinate system to align exactly with the axes of maximum variance in the dataset. The fundamental equation of eigen-decomposition is:

$$C_{MCD} \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

For a 17×17 covariance matrix, this extracts 17 pairs of eigenvalues (λ_i) and eigenvectors ($\mathbf{v}_i$):

- **The Eigenvectors ($\mathbf{v}_i$):** These are the directional vectors. In football terms, each eigenvector represents a distinct tactical profile (e.g., "Possession Dominance" or "Defensive Action"). The values inside the vector dictate the recipe of stats required to execute that profile.
- **The Eigenvalues (λ_i):** These represent magnitude. Specifically, the eigenvalue defines the exact amount of variance that its corresponding tactical profile explains within the dataset.

The engine sorts these components in descending order by eigenvalue, ranking the tactical profiles by statistical importance. It then applies dimensional truncation, deliberately retaining only the top 9 components ($k = 9$).

This truncation is a critical filtering mechanism. The top 9 components account for approximately 85% of the dataset's total tactical variance. The remaining 15% is discarded because the lower-ranked components represent statistical noise - highly specific, mathematically unstable correlations that do not consistently define player performance.

4. Vector Reassembly and Normalisation

At this stage, the engine holds a 17×9 matrix (17 stats distributed across 9 independent tactical profiles). It must collapse this matrix back into a single 1D positional weight vector.

For every stat (j), the engine iterates through the 9 retained components. It extracts the stat's internal weight within the component ($v_{j,i}$), and scales it by the square root of the component's eigenvalue ($\sqrt{\lambda_i}$).

Two mathematical decisions here are paramount:

1. **Absolute Values ($|v|$):** The sign (positive/negative) of an eigenvector is mathematically arbitrary; it only indicates direction along the axis. Taking the absolute value ensures the algorithm captures the true *magnitude* of the stat's influence on the tactical profile.
2. **Square Root Scaling ($\sqrt{\lambda}$):** Because eigenvalues (λ) represent variance, taking the square root converts them to standard deviation. Scaling the weights by standard deviation accurately reflects the component's actual physical spread in the data space.

The mathematical aggregation to find the raw weight (w_{raw}) for a specific stat (j) is:

$$w_{raw,j} = \sum_{i=1}^k |v_{j,i}| \sqrt{\lambda_i}$$

The exact Python implementation mapping to this logic:

```
# Scale absolute eigenvectors by the square root of their explained
variance
weighted_loadings = np.abs(top_eigenvectors) * np.sqrt(top_variances)
# Sum across the components to create a 1D vector (length 17)
raw_weights = np.sum(weighted_loadings, axis=1)
# Normalize the final vector to sum to 1.0
final_weights = raw_weights / np.sum(raw_weights)
```

Finally, the engine normalises the 17 raw weights by dividing each by the total sum of the array. This yields the final Positional Weight Vector, which sums to 1.0 and accurately reflects the multidimensional nature of the role without double-counting the underlying telemetry.

V. The Scoring Algorithm

With the live match telemetry mathematically standardised into contextual Z-scores (Section III) and the ideal tactical philosophies distilled into independent Positional Weight Vectors (Section IV), the Analytics Engine reaches its core synthesis phase.

At this stage, the live application dynamically queries the hardcoded PCA weight matrices and aligns them with the player's live performance data. By calculating the mathematical dot product of these two arrays - multiplying every individual standardised metric by its mathematically derived tactical weight, and summing the result—the engine compresses 17 disparate dimensions of telemetry into a singular, unbounded "Raw Performance Score."

However, an unbounded raw score (e.g., +4.82 or −1.75) is unreadable to an end user who requires the traditional 1.0 to 10.0 football rating scale. Furthermore, a purely statistical dot product lacks the macro-context of the match result. The engine cannot award a defender who executes flawless tackles a perfect rating if the team simultaneously concedes four goals.

To convert this raw synthesis into a mathematically bounded, contextually aware final rating, the engine passes the raw score through a rigorous three-step pipeline: positional context modification, logistic (sigmoid) transformation, and macro-supremacy scaling.

Positional Modifiers & Contextual Heuristics

The foundational step of the scoring algorithm is the dot product: multiplying the 17 standardised Z-scores by their respective positional weights to generate a base score. However, while a dot product is perfectly linear, football is inherently non-linear and highly contextual. A purely statistical synthesis cannot account for the asymmetrical responsibilities of different tactical roles, nor can it comprehend the macro-state of the match. To bridge the gap between pure mathematics and football reality, the raw score is passed through a rigid pipeline of positional modifiers.

This modification pipeline executes four distinct architectural layers:

- 1. Asymmetric Flooring:** Standardised Z-scores mathematically punish players for inaction. The engine intercepts this by applying positional floors. For example, a Centre Back is granted an "Attacking Floor" that neutralises negative Z-scores for a lack of shots or assists, effectively treating their offensive absence as a neutral baseline (0.0) rather than a mathematical detriment.
- 2. Efficiency Safeguards:** Because percentage-based stats (like pass accuracy) can generate massive negative Z-scores from tiny sample sizes (e.g., completing 1 of 3 passes), the engine caps negative efficiency variance. A poor success rate in isolated actions acts as a standard penalty rather than creating an unrecoverable mathematical black hole.
- 3. Absolute Output Multipliers:** Z-score standardisation is designed to smooth out variance, but goals and assists are absolute, match-defining events. Therefore, these events are

applied as flat numerical bonuses *after* the dot product calculation, ensuring that direct goal contributions always yield a massive rating spike regardless of the underlying statistical distribution.

4. **Contextual Heuristics:** The engine deploys complex, position-specific logic to evaluate how a player accumulated their stats. It does not just measure volume; it measures efficiency ratios and match context. For example, the algorithm calculates whether a striker is acting as an offensive "black hole" by comparing their turnover rate to their positive involvements, or dynamically scales a defender's clean sheet bonus based on the quality of chances (xG) the opponent created.

Deep Dive: The Synthesis Pipeline & Contextual Heuristics >

05a - Synthesis Pipeline & Contextual Logic

Phase I: Pre-Synthesis Matrix Bounding

Before the engine calculates the baseline match rating, it must conditionally bind the raw Z-score input vector to prevent isolated data points from collapsing the linear algebra.

The engine iterates through the 17-element telemetry array and applies rigid minimum thresholds depending on the player's position.

- **Efficiency Safeguards:** Percentage-based metrics (e.g., `pass_accuracy_z`) are strictly floored at -0.75 . This ensures a 0% success rate on a tiny sample size functions as a standard "bad day" penalty rather than a multi-sigma black hole.
- **Tactical Instruction Floors (e.g., Fullbacks):** To protect players from being mathematically punished for following managerial orders, the engine applies contextual forgiveness. For example, a Fullback instructed to "Stay Back While Attacking" will naturally log zero dribbles and low Expected Threat (xT). The engine intercepts these specific tactical metrics and floors them at -0.50 (a microscopic penalty), ensuring a defensive fullback isn't mathematically destroyed for ignoring the final third.
- **Metric Normalisation (e.g., Central Midfielders):** For positions where goals are rare, anomalous per-90 goalscoring can artificially inflate the Z-score to extreme levels (e.g., $+4.0$). The engine caps the standardised impact of goals and assists for CMs at $+1.5$ standard deviations, preventing statistical runaway.

Phase II: Base Synthesis (The Dot Product)

With the input vector safely bounded, the raw foundation of the match rating is calculated using the dot product of the modified Z-scores and the mathematically derived Positional Weight Vector.

Utilising NumPy's optimised C-backend, the engine arrays the metrics and computes the linear combination:

$$S_{raw} = \sum_{i=1}^{17} (Z_i \times W_i)$$

```
# Fast, vectorized linear combination of the 17 core metrics
return np.dot(
    np.array([z_scores.get(f"{col}_z", 0) for col in col_names]),
    weights,
)
```

Phase III: Post-Synthesis Contextual Heuristics

Following the dot product, the engine applies non-linear modifiers to the resulting S_{raw} score to account for complex match context, flat output rewards, and dynamic player archetypes.

Dynamic Archetype Branching (Fullbacks):

Rather than forcing players into a single rigid positional mould, the engine dynamically rewards different playstyles within the same position. For example, one can look at fullbacks. For these players, the algorithm branches into two potential reward paths based on their statistical output:

- **The "Third CB" Archetype:** If a tucked-in or defensive fullback registers elite defensive volume ($Z > 1.5$ for tackles and possession won), they receive a scaling defensive bonus.
- **The "Express Train" Archetype:** Conversely, if an overlapping fullback registers elite physical exertion ($Z > 1.5$ for distance covered) combined with high xT generation, they trigger a completely separate attacking progression bonus.

Striker Efficiency Penalties & Synthetic xG:

The engine evaluates Striker efficiency using an embedded synthetic Expected Goals (xG) model, assigning a static baseline value of **0.20 xG per shot**.

This hardcoded constant is explicitly calibrated to bypass two critical data flaws. First, EA FC's post-match telemetry only provides aggregate Team xG; individual player xG is entirely missing from the dataset, and naively dividing Team xG among attackers is mathematically unsound. Second, the game's native xG algorithm is demonstrably inflated compared to real-world probability models (e.g., *Opta* or *StatsBomb*). The native engine frequently assigns statistically impossible values - such as 0.90 to 1.00 xG for standard chances, or 0.60 to 0.70 xG for low-percentage shots from the edge of

the box. By establishing a flat 0.20 baseline, the Analytics Engine explicitly rejects the game's flawed probability model and enforces a rigorous, realistic shot average for an elite Striker.

$$\text{Deficit} = (\text{Shots} \times 0.20) - \text{Goals}$$

If a Striker takes high volume (> 3 shots) but generates a deficit greater than 0.75, the engine applies a scaling penalty (capped at -0.8). This penalty accurately punishes players who demand the ball but consistently waste high-value opportunities, neutralising their rating.

Absolute Output & Clean Sheet Scaling:

Finally, the engine applies macro-contextual rewards. Direct goal contributions, which were bounded during Phase I, are reintroduced as flat, absolute multipliers (e.g., $+0.8$ for a CM goal). Furthermore, defensive clean sheet bonuses are dynamically scaled by the opponent's total xG. A clean sheet against 2.0 opponent xG yields a minimal $+0.15$ reward (indicating luck or exceptional goalkeeping), while a clean sheet against 0.5 xG yields a massive $+0.50$ reward (indicating true defensive dominance).

Logistic Mapping

After the positional modifiers are applied, the engine finally has a single raw score for the player's performance. However, because this score is based on standard deviations, it looks like a bizarre decimal (e.g., a terrible game might be -2.5 , an average game 0.0 , and a world-class game 4.8). This format is useless to an end-user. The application must convert this value into the classic 1.0 to 10.0 match rating scale used by *FIFA*, *WhoScored*, and football fans worldwide.

A naïve approach would be to scale the numbers evenly - mapping the highest historical score to a 10 and the lowest to a 1. However, football throws up crazy outliers. If a player somehow scores 6 goals in a single game, an even scale would permanently warp the system. That 6-goal game would become the new 10.0, and every normal, excellent performance would be crushed down to a 4.0.

To safely lock the ratings between 1 and 10 without breaking the game, the Analytics Engine uses an "S-Curve" (mathematically known as a Sigmoid transformation).

Instead of stretching the numbers evenly, the curve is anchored to a psychological baseline: an exactly average statistical performance is locked to a **6.0 rating**. In the middle of the curve - where 95% of normal football matches happen - the scoring is highly sensitive, easily separating a decent 6.5 performance from a great 7.5. However, at the extreme top end, the curve applies "diminishing returns." A world-class hat-trick might earn a 9.5, and adding a fourth

goal might only push it to a 9.8. This ensures that no matter how unbelievable a player's stats become, the rating gracefully slows as it approaches 10.0, never breaking the ceiling.

Deep Dive: Logistic Mapping Mathematics >

05b - Logistic Mapping Mathematics

1. The Inverse Logistic Function

To map the unbounded continuous variable $S_{raw} \in (-\infty, \infty)$ to the recognised 0.0 – 10.0 continuum, the engine utilises a scaled inverse logistic function:

$$R_{raw} = 10 \times \left(\frac{1}{1 + e^{-k(S_{raw} - S_0)}} \right)$$

- R_{raw} : The resulting 0 – 10 match rating.
- S_{raw} : The input unbounded Z -score sum.
- k : The steepness parameter (controlling the slope of the S-curve).
- S_0 : The translation parameter (shifting the curve to align the median).

2. Anchoring the Baseline (S_0 and the 6.0 Median)

In a pure mathematical model mapped from 0 to 10, the median would default to 5.0. However, the Analytics Engine explicitly rejects a 5.0 median because it violates the psychological reality of football ratings.

In established industry frameworks (e.g., *L'Equipe*, WhoScored), ratings possess a natural positive skew. A player who performs terribly rarely receives below a 4.0, whereas excellent players frequently receive 8.0+. To account for this asymmetric headroom, the engine sets the baseline for an entirely average, 0.0 standard deviation performance to exactly **6.0**.

To achieve this, the engine mathematically solves for the translation parameter S_0 :

$$\begin{aligned} 6.0 &= 10 \times \left(\frac{1}{1 + e^{k \cdot S_0}} \right) \\ 1 + e^{k \cdot S_0} &= \frac{10}{6.0} \approx 1.666 \implies e^{k \cdot S_0} = \frac{2}{3} \\ k \cdot S_0 &= \ln \left(\frac{2}{3} \right) \approx -0.405 \end{aligned}$$

3. Calibrating the Steepness Parameter ($k = 0.85$)

With the translation baseline established, the developer must calibrate the steepness parameter (k). This constant dictates how aggressively the rating climbs or falls as standard deviations accumulate.

If k is set too high (e.g., $k = 1.5$), the S-curve is too vertical; a standard great game ($+2.5\sigma$) would instantly hit a 10.0, making the perfect rating far too common. If k is set too low (e.g., $k = 0.4$), the curve is too flat; a historic, multi-goal anomaly ($+4.0\sigma$) would mathematically struggle to surpass an 8.5, unfairly punishing the user for a world-class performance.

The engine hardcodes the steepness parameter at $k = 0.85$ as the optimal "Goldilocks" threshold. This specific slope mathematically protects the mythical 10.0 rating while providing accurate scalar rewards. Under $k = 0.85$, highly anomalous $+3\sigma$ events organically approach a 9.5 rating. To achieve a true 10.0, a player must push the absolute limits of the statistical distribution without hitting a rigid mathematical ceiling too early.

Solving for S_0 with $k = 0.85$ yields $S_0 \approx -0.477$. Thus, the final algebraic formula embedded in the engine is:

$$R_{raw} = 10 \times \left(\frac{1}{1 + e^{-0.85(S_{raw} + 0.477)}} \right)$$

4. Algorithmic Implementation

The Python implementation explicitly calculates S_0 at runtime using `np.log`, preventing floating-point rounding errors that could shift the 6.0 baseline over thousands of iterations.

```
def _apply_sigmoid_transformation(self, raw_score: float) -> float:
    # Steepness parameter calibrated for 3-sigma events hitting ~9.5
    k: float = 0.85

    # Calculate the exact translation parameter for a 6.0 median
    s_0: float = np.log(2 / 3) / k

    return 10 * (1 / (1 + np.exp(-k * (raw_score - s_0))))
```

VI. Macro-Context, Hybridisation & Goalkeepers

While the logistic curve successfully synthesises individual telemetry into a recognisable 1-to-10 rating, evaluating a player purely in a statistical vacuum is inherently flawed. Football is not a static environment. The match's overall momentum deeply influences a player's individual

metrics; their tactical role can change depending on the phase of play, and specific positions operate under entirely different physical and data rules.

To finalise the match rating, the Analytics Engine applies three macro-resolutions. It introduces a team-level environment scalar, deploys a multi-positional hybridisation formula for players who shift roles mid-match, and completely isolates Goalkeepers into their own bespoke analytical pipeline.

The Match Supremacy Scalar

In football, it is significantly easier to string together 50 passes and register an assist when your team has 70% possession and is pinning the opponent in their own box. Conversely, a centre-back on a team facing an absolute onslaught will naturally accumulate massive tackling and clearance numbers simply because the ball is always near them.

Statistical research into football telemetry has proven that evaluating a player in a vacuum is flawed. In fact, roughly 26% of a player's statistical variance is dictated entirely by how dominant their team was on the day^[4]. If an algorithm ignores this, it will artificially inflate the ratings of players on dominant teams (stat-padding) while systematically punishing players on struggling teams.

To neutralise this environmental bias, the Analytics Engine applies a final macro-adjustment to the 1-to-10 score: **The Match Supremacy Scalar**.

The engine calculates which team genuinely dominated the match by comparing the two teams' Expected Goals (xG). Based on this ratio, the engine applies a bidirectional adjustment:

- **The Inflation Penalty:** If a player's team heavily dominated the match, the engine applies a slight penalty to their final rating. This strips away the "easy game" padding and ensures their 9.0+ rating was actually earned through individual brilliance, not just team superiority. (To prevent unfairly punishing elite teams, this penalty is strictly capped).
- **The Siege Bonus:** If a player's team was heavily outplayed, the engine applies a rating boost. This rewards players who maintained a solid baseline output despite being tactically besieged, recognising their performance against overwhelming odds.

Deep Dive: The Match Supremacy Scalar >

06a - The Match Supremacy Scalar

1. The Hierarchical Variance Problem

The architectural necessity of the Match Supremacy Scalar is derived from the statistical properties of invasion-territorial sports. Football telemetry is inherently

hierarchical; a player's individual statistical output is deeply nested within the macro-performance of their team.

Recent peer-reviewed statistical analyses utilising Generalised Linear Mixed Models (GLMMs) to deconstruct commercial rating systems revealed an Intraclass Correlation Coefficient (ICC) of approximately 0.26. This indicates that 26% of the variance in professional football ratings is entirely due to macro-level hierarchical contexts, rather than the isolated actions of the player. Without a structural modifier, the algorithm would fail to separate individual brilliance from team dominance. The Match Supremacy Scalar is designed to explicitly neutralise this covariance, realigning the final output with true individual merit.

2. Expected Goals Delta and Laplace Smoothing

To quantify hierarchical team dominance, the Analytics Engine calculates the Expected Goals Ratio (Δ_{xG}) between the player's team and the opponent.

However, raw division introduces a critical mathematical vulnerability: if a team completely stifles the opposition, resulting in 0.0 opponent xG, the division by zero collapses the algorithm. To resolve this, and to dampen the volatility of low-event matches (e.g., 0.2 xG vs 0.1 xG), the engine applies Laplace Smoothing, adding a +1.0 constant to both the numerator and the denominator:

$$\Delta_{xG} = \frac{\text{Team xG} + 1}{\text{Opponent xG} + 1}$$

3. Logarithmic Dampening

Because team dominance operates on a curve of diminishing returns (the tactical difference between 1.0 and 2.0 xG is massive, whereas the difference between 4.0 and 5.0 xG is negligible), the engine passes the smoothed ratio through a natural logarithm ($\ln$).

The logarithm is scaled by a dampening parameter ($\gamma = 0.2$) to compress the output into a tight numerical boundary suitable for the 1-to-10 rating scale:

$$\text{Scalar} = 0.2 \times \ln(\Delta_{xG})$$

4. Application and Asymmetrical Capping

The resulting scalar is mathematically **subtracted** from the player's logistic raw rating, creating a bidirectional contextual adjustment:

- **The Inflation Penalty ($\Delta_{xG} > 1$):** If a player's team dominates, the natural log evaluates to a positive number. Because it is subtracted from the final rating, it acts as a penalty, stripping away artificial inflation.
- **The Siege Bonus ($\Delta_{xG} < 1$):** If a player's team is battered, the natural log evaluates to a negative number. Subtracting a negative results in a positive, actively boosting the rating.

The algorithm explicitly caps the positive scalar at 0.25. This ensures the "Inflation Penalty" cannot deduct more than a quarter of a point from a player, protecting elite performers from unjust punishment for their team's excellence. The "Siege Bonus," however, is left organically uncapped to reward defensive masterclasses against overwhelming odds properly.

Multi-Positional Hybridisation

Modern football is increasingly positionless. Tactical fluidity often dictates that players change roles mid-match - a fullback may invert into central midfield, or a winger may transition into a second striker, depending on the game state.

However, evaluating these hybrid performances introduces a major telemetry constraint. When a user enters multiple positions for a player (e.g., "CB, RB" in the UI), the Analytics Engine only has aggregated match stats. The data is "blind" - the engine does not know if the player spent 80 minutes at CB and 10 minutes at RB, nor does it know which tackles occurred in which role.

To resolve this, the engine embraces parallel evaluation. It uses the player's entire unpartitioned match data and runs it through the independent weight matrices for each entered position, generating multiple distinct match ratings.

Collapsing these multiple ratings into a single final score presents an architectural challenge:

- **The Accountability Void (Maximum):** The engine cannot simply award the highest rating and discard the rest. If a player puts in a world-class performance in their main role but performs disastrously when asked to cover a secondary role for a portion of the match, they must be held accountable for that poor stint. Ignoring the lower rating entirely grants them an unearned free pass.
- **The Versatility Punishment (Average):** Conversely, the engine cannot use a simple average. If a player earns a 9.0 in their primary role but a 6.0 in the secondary role due to unfamiliarity or lack of volume, a pure average (7.5) unjustly destroys their rating, punishing them far too heavily for stepping in to fill a tactical gap.

To bridge this gap, the engine deploys a proprietary **Alpha Drag** system. The algorithm assumes the player's highest calculated rating reflects their primary tactical role. It anchors the

final score to this maximum rating, but applies a fractional "drag" based on the lower scores. This ensures the player is rightfully penalised for poor secondary performances, but the punishment is scaled down so it does not erase their primary success.

Deep Dive: The Alpha Drag Coefficient >

06b - Multi-Position Alpha Drag

1. The Telemetry Constraint (Blind Aggregation)

When a player plays multiple tactical roles in a single match, the engine receives an unpartitioned telemetry array (T_{match}). Because the system lacks temporal parsing (minutes played per position) and spatial parsing (event locations), it cannot isolate which specific statistics belong to which tactical role.

To bypass this blind aggregation, the algorithm executes Parallel Matrix Evaluation. It processes T_{match} through n distinct Positional Weight Vectors corresponding to the user's input, generating a set of positional ratings $R = \{r_1, r_2, \dots, r_n\}$.

2. The Alpha Drag Formula

To synthesise the set R into a single scalar without falling victim to maximum-value loopholes or punitive averaging, the engine identifies the maximum rating (R_{max}) and the arithmetic mean of all ratings (R_{mean}).

It then applies the Alpha Drag Coefficient (α):

$$R_{hybrid} = R_{max} - \alpha(R_{max} - R_{mean})$$

3. Calibrating the Alpha Coefficient ($\alpha = 0.25$)

The engine hardcodes the drag coefficient at $\alpha = 0.25$.

This specific constant acts as a weighted centre of gravity. By subtracting exactly 25% of the distance between the maximum and mean ratings, the algorithm effectively assigns a **75% confidence weight** to the player's highest tactical alignment. The underlying mathematical assumption is that the highest rating naturally correlates with the position where the player spent the vast majority of their minutes and accumulated the bulk of their statistics.

```
r_max: float = max(calculated_ratings)
r_mean = np.mean(calculated_ratings)

# Apply a 25% drag toward the mean
```

```
alpha = 0.25  
hybrid_rating = r_max - (alpha * (r_max - r_mean))
```

The remaining 25% "drag" serves as a mathematical tax for statistical bleed. If a Centre Back (CB) plays a fraction of the match at Right Back (RB), their aggregate passing and tackling stats are evaluated through the RB matrix, which expects crosses and high progressive distance. This inevitably generates a lower secondary rating. The $\alpha = 0.25$ parameter ensures this secondary rating slightly anchors the R_{max} , creating a realistic, synthesised representation of a fragmented tactical performance without triggering an unrecoverable plunge in the final match rating.

Goalkeeper Isolation Heuristics

The structural realities of the goalkeeper position render standard analytics algorithms entirely obsolete. This is not merely a tactical difference; it is a fundamental data constraint. For outfield players, the Analytics Engine receives a 17-metric telemetry matrix (measuring progression, passes, distance covered, xT, etc.). For goalkeepers, the engine receives a completely different, highly specialised 11-metric matrix consisting purely of reactive events (e.g., *shots against*, *saves*, *rush saves*, *punch saves*, *penalty metrics*).

Attempting to force an 11-metric reactive array through a 17-dimensional PCA weight matrix designed for outfielders would cause the linear algebra to collapse mathematically. Therefore, the engine completely isolates goalkeepers from the outfield architecture, routing them into a bespoke, dedicated heuristic pipeline.

The core of this pipeline is **Expected Goals Prevented (xGP)**. Rather than simply counting raw saves, the engine calculates the opponent's total Expected Goals and subtracts the actual goals conceded. This creates a true, context-aware measurement of shot-stopping value.

Furthermore, the goalkeeper pipeline deliberately reverses several contextual heuristics used for outfielders to reflect their reactive role. The most prominent example is the "Inverted Clean Sheet Bonus." When a central defender keeps a clean sheet against a team that generated 0.5 xG, the defender receives a massive rating boost because they actively smothered the opponent. However, if a goalkeeper keeps a clean sheet against 0.5 xG, it indicates they were largely a "passenger" in a game dominated by their own defence, yielding a minimised bonus. Conversely, if the defence collapses and allows 2.5 xG, but the goalkeeper still secures a clean sheet, they trigger a "Bailout" bonus and receive a massive rating spike for single-handedly carrying the team.

06c - Goalkeeper Heuristics

1. Matrix Dimensionality Mismatch

Outfield ratings are computed as a 17-element linear combination (dot product) of standardised Z-scores, mapped against a 17×9 PCA weight matrix. Goalkeeper telemetry is fundamentally incompatible with this architecture, as the engine receives a structurally distinct 11-element array restricted entirely to reactive metrics (`shots_against` , `shots_on_target` , `saves` , `goals_conceded` , `save_success_rate` , `punch_saves` , `rush_saves` , `penalty_saves` , `penalty_goals_conceded` , `shoot_out_saves` , `shoot_out_goals_conceded`).

To resolve this dimensionality mismatch, the Analytics Engine routes goalkeepers to a dedicated `calculate_gk_rating` method. This method derives a standalone mathematical heuristic (H_{GK}), standardises it against historical goalkeeper distributions, and applies specific reactive floors.

2. The Core Heuristic Formulation

The base performance of a goalkeeper is captured via a custom synthetic formula heavily weighted toward Expected Goals Prevented (xGP), where $xGP = \text{Opponent xG} - \text{Goals Conceded}$.

$$H_{GK} = (xGP \times 1.5) + \left(\ln(\text{Saves} + 1) \times \frac{\text{Save}\%}{100} \right)$$

Mathematical Justifications:

- **xGP Weighting (1.5):** By anchoring the formula to xGP , the engine prioritises shot quality over shot volume.
- **Logarithmic Volume Dampening:** By passing the raw save count through a natural logarithm $\ln(\text{Saves} + 1)$, the algorithm mathematically dampens the impact of facing a high volume of low-quality shots.
- **Efficiency Scaling:** The logarithmic volume is multiplied by the save percentage, heavily punishing goalkeepers who make numerous saves but still concede highly preventable goals.

Once H_{GK} is calculated, it is standardised into a familiar Z-score using a pre-calculated, historical Goalkeeper mean and standard deviation:

$$S_{raw} = \frac{H_{GK} - \mu_{GK}}{\sigma_{GK}}$$

3. Contextual Bounding & Low-Volume Forgiveness

Because goalkeeper stats are prone to extreme small-sample-size anomalies, the engine applies rigorous pre-synthesis bounding to S_{raw} .

- **The Spectator Exemption:** If a goalkeeper faces 0 shots on target and concedes 0 goals, their raw score defaults to exactly 0.0 (resulting in the 6.0 baseline rating).
- **Low-Volume Forgiveness:** In football, a goalkeeper may face only 1 shot all game and concede 1 goal, resulting in a mathematically catastrophic 0% save rate. To prevent the rating from plummeting to a 3.0 or 4.0, the engine applies the "Match 69 Fix." If a keeper faces ≤ 3 shots and concedes ≤ 1 goal, their raw Z-score is rigidly floored at -0.5 , mitigating the small-sample penalty.

4. The Inverted Clean Sheet Bonus

While defenders are rewarded most for clean sheets against *low* xG (indicating defensive control), goalkeepers are rewarded most for clean sheets against *high* xG (indicating a goaltending bailout). The engine implements this through conditional branching:

```
# 1. Passenger: Low threat, keeper didn't have to do much
if xg_against <= 1.0 and xgp < 0.95:
    return raw_score + 0.30

# 2. Bailout: High threat, keeper made a ton of saves
elif xg_against > 2.0 and saves >= 5:
    return raw_score + 0.80

# 3. Standard: Any other clean sheet scenario
else:
    return raw_score + 0.50
```

By inverting the contextual logic from the outfield algorithms, applying absolute bonuses for penalty saves ($+0.5\sigma$ per save), and finally applying the Match Supremacy Scalar, the engine finalises a highly accurate, context-aware 1-to-10 rating for the goalkeeper.

VII. Conclusion

A tension between statistical objectivity and tactical reality has historically hampered the quantification of football performance. Commercial rating systems and native in-game algorithms often rely on rigid, one-size-fits-all linear models. These models fail to account for asymmetric responsibilities across positions, aggressively punish players for executing specific

managerial instructions, and unquestioningly reward players for hierarchical dominance within their team.

The custom Analytics Engine detailed in this whitepaper was engineered to resolve these epistemological flaws. By standardising raw telemetry into variance-aware Z-scores (Section III), the engine creates a mathematically levelled playing field across all metrics. By applying the Minimum Covariance Determinant and Eigen-Decomposition (Section IV), the algorithm actively hunts for the true tactical DNA of every position, filtering out statistical noise and deriving perfect Positional Weight Vectors without human bias.

Most importantly, the engine wraps this linear algebra in rigorous, context-aware heuristics (Sections V & VI). By integrating synthetic xG models, the Alpha Drag hybridisation coefficient, and the GLMM-backed Match Supremacy Scalar, the algorithm bridges the gap between a spreadsheet and the pitch.

The resulting output is a robust, dynamic 1.0 to 10.0 rating system that does not merely count what a player did, but mathematically evaluates *how* and *why* they did it. It successfully mimics the subjective, nuanced eye of an elite football analyst while maintaining the strict, reproducible objectivity of a data science framework.

Appendix: Complete Positional Heuristics Reference >

Appendix A - Positional Heuristics

This appendix serves as the complete technical glossary and tuning manual for all outfield roles.

(Note: Unless otherwise specified, an "Efficiency Floor" is applied universally to `tackle_success_rate`, `pass_accuracy`, `dribble_success_rate`, and `shot_accuracy`, capping negative variance strictly at -0.75σ .)

1. Attackers (ST, RW, LW)

- **Striker (ST):**
 - *Ghosting Forgiveness:* Goals, assists, and shots are floored at -2.0σ .
 - *Offside Leniency:* Offsides are floored at -1.5σ .
 - *Direct Output Multipliers:* Raw goals are multiplied by $+1.2$, and assists by $+0.6$.
 - *Complete Forward Bonuses:* Elite passing ($> 2.0\sigma$) yields $+0.20$ per σ over the threshold. Elite dribbling ($> 2.0\sigma$) yields $+0.20$ per σ .
 - *Wasteful Finisher Penalty:* Assigns 0.20 xG per shot. If a striker takes > 3 shots and generates a finishing deficit > 0.75 , a penalty of $0.25 \times \text{Deficit}$ is applied (capped at -0.8).

- *Black Hole Penalty*: If possession is lost > 4 times with a turnover-to-involvement ratio > 1.5 , a penalty of $0.08 \times \text{Excess Losses}$ is applied (capped at -0.6).
- *Target Man Bonus*: If positive involvements are ≥ 15 and retention ratio is > 4.0 , a bonus of $0.02 \times \text{Excess Retention}$ is applied (capped at $+0.4$).
- **Wingers (RW/LW)**:
 - *Defensive Absolution*: Tackles, tackle success, and possession won are floored at -0.5σ .
 - *Detriment Floors*: Fouls committed floored at -1.5σ ; offsides floored at -2.0σ .
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.8$, assists by $+0.6$.
 - *Elite Outlier Bonuses*: Dribbles $> 2.0\sigma$ yield $+0.25$ per σ . xT $> 2.0\sigma$ yields $+0.20$ per σ . Passes $> 2.0\sigma$ yield $+0.20$ per σ .
 - *High Press Bonus*: Tackles $> 1.5\sigma$ yield $+0.10$ per σ .
 - *Wastefulness Penalty*: If a winger registers ≥ 3 shots and 0 goals, a penalty of -0.10 per shot over 2 is applied.

2. Midfielders (CAM, CM, CDM, WM)

- **Central Attacking Midfielder (CAM)**:
 - *Defensive Exemption*: Tackles, tackle success, and possession won floored at -0.5σ .
 - *Passenger Floor*: Passes, dribbles, shots, distance covered, sprint distance, and xT floored at -1.0σ .
 - *Detriment Floor*: Fouls, possession lost, and offsides floored at -1.5σ .
 - *Z-Score Capping*: Standardized goals and assists cannot exceed $+2.0\sigma$.
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.7$, assists by $+0.9$.
 - *Maestro Bonus*: Passes $> 1.0\sigma$ AND xT $> 1.5\sigma$ yield a flat $+0.40$.
 - *Shadow Striker Bonus*: Shots $> 1.5\sigma$ yield $+0.10$ per σ .
 - *Risky Creator Bonus*: Pass accuracy $> 1.0\sigma$ AND possession lost $> 1.0\sigma$ yield a flat $+0.15$.
 - *Modern 10 Bonus*: Tackles $> 1.0\sigma$ AND possession won $> 1.0\sigma$ yield a flat $+0.50$.
- **Central Midfielder (CM)**:
 - *Attacking Floor*: Goals, assists, shots, and shot accuracy floored at 0.0σ .
 - *Z-Score Capping*: Standardized goals and assists cannot exceed $+1.5\sigma$.
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.8$, assists by $+0.6$.
 - *Box-to-Box Scaling*: Tackles $> 2.0\sigma$ yield $+0.20$ per σ . Possession won $> 2.0\sigma$ yields $+0.40$ per σ . Passes $> 2.0\sigma$ yield $+0.40$ per σ . Dribbles $> 2.0\sigma$ yield $+0.20$ per σ .

- *Clean Sheet Bonus*: Flat $+0.15$ if the team concedes 0 goals and the player logged ≥ 60 minutes.
- **Central Defensive Midfielder (CDM):**
 - *Attacking Floor*: Goals, assists, shots, and shot accuracy floored at 0.0σ .
 - *Detriment Floor*: Fouls committed floored at -1.0σ .
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.5$, assists by $+0.4$.
 - *Defensive Dominance*: Tackles $> 2.0\sigma$ yield $+0.25$ per σ . Possession won $> 2.0\sigma$ yields $+0.25$ per σ .
 - *Passing Prowess*: Passes $> 2.0\sigma$ yield $+0.35$ per σ .
 - *Reliable Shift*: If minutes ≥ 60 AND possession lost is 0, yields a flat $+0.20$.
 - *Clean Sheet Bonus*: Flat $+0.20$ if minutes ≥ 60 AND 0 goals conceded.
- **Wide Midfielder (RM/LM):**
 - *Attacking Floor*: Goals, assists, shots, and shot accuracy floored at -0.5σ .
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.6$, assists by $+0.8$.
 - *Two-Way Engine Bonus*: Passes $> 1.0\sigma$ AND tackles $> 1.0\sigma$ yield a flat $+0.40$.
 - *Wide Progression Bonus*: $xT > 1.5\sigma$ yields $+0.15$ per σ .

3. Defenders (CB, FB/WB, LWB/RWB)

- **Center Back (CB):**
 - *Attacking Floor*: Goals, assists, shots, and shot accuracy floored at 0.0σ .
 - *Professional Foul Leniency*: Fouls committed floored at -2.0σ .
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.6$, assists by $+0.4$.
 - *Dominance Bonus*: Tackles $> 2.0\sigma$ yield $+0.25$ per σ . Possession won $> 2.0\sigma$ yields $+0.25$ per σ . Passes $> 1.5\sigma$ yield $+0.30$ per σ .
 - *Dynamic Clean Sheet*: Bonus scales inversely with opponent xG. Max bonus ($+0.5$) for ≤ 1.0 xG. Min bonus ($+0.15$) for ≥ 2.0 xG. Standard bonus ($+0.35$) for all other clean sheets.
 - *Collapse Penalty*: Flat -0.3 penalty if the team concedes ≥ 3 goals and the player logged ≥ 60 minutes.
- **Fullback (LB/RB):**
 - *Attacking Floor*: Goals, assists, shots, and shot accuracy floored at 0.0σ .
 - *Tactical Instruction Floor*: Dribbles, xT, distance covered, and sprint distance floored at -0.50σ .
 - *Foul Leniency*: Fouls committed floored at -1.5σ .
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.4$, assists by $+0.6$.
 - *The "Third CB" Archetype*: Tackles $> 1.5\sigma$ yield $+0.15$ per σ . Possession won $> 1.5\sigma$ yields $+0.15$ per σ .

- *The "Express Train" Archetype*: Distance covered $> 1.5\sigma$ yields $+0.10$ per σ . Sprint distance $> 1.5\sigma$ yields $+0.10$ per σ . xT $> 1.5\sigma$ yields $+0.15$ per σ .
- *Creativity Bonus*: Passes $> 2.0\sigma$ yield $+0.15$ per σ . Dribbles $> 1.5\sigma$ yield $+0.15$ per σ .
- *Clean Sheet / Collapse*: Identical to CB logic.
- **Wingback (LWB/RWB)**:
 - *Attacking Floor*: Goals, assists, shots, and shot accuracy floored at 0.0σ .
 - *Active Detriment Cap*: Fouls and possession lost floored at -1.5σ .
 - *Passenger Cap*: Passes, dribbles, tackles, possession won, distance covered, sprint distance, and xT floored at -1.0σ .
 - *Direct Output Multipliers*: Raw goals are multiplied by $+0.6$, assists by $+0.8$.
 - *Physical / Progression Bonuses*: Distance covered $> 1.5\sigma$ yields $+0.15$ per σ . Sprint distance $> 1.5\sigma$ yields $+0.15$ per σ . xT $> 1.5\sigma$ yields $+0.20$ per σ .
 - *Two-Way Synergy*: Tackles $> 1.0\sigma$ AND possession won $> 1.0\sigma$ yield a flat $+0.35$.
 - *Clean Sheet*: Identical to CB logic.

Footnotes

1. Drs Raffaele Poli, Loïc Ravenel and Roger Besson (2021). Monthly Report 68: Analysis of distances covered in professional football. *CIES Football Observatory* [Available Here](#) ↩
2. Kubayi A. (2019). Evaluation of match-running distances covered by soccer players during the UEFA EURO 2016. *South African journal of sports medicine*, 31(1), v31i1a6127. [Available Here](#) ↩
3. Matthew Herring-Rogers. Distance in football matters: Use it wisely. *PlayerData*. [Available Here](#) ↩
4. James Ball, Minh Huynh and Matthew C. Varley (2025). Comparing player rating systems as a metric for assessing individual performance in soccer. *Journal of Sports Sciences* [Available Here](#) ↩